Development of an AI-Powered Intelligent Document Processing and Administrative Automation Assistant for SMEs

Senior Project — Progress Report

Karam Masry
Submitted to: dr. Issam mosallem
September 15, 2026

1 Introduction

This report is meant to give an honest picture of where the project stands right now, roughly a month into active development. The original proposal set out to build a web application that automates three concrete administrative pain points for small and medium enterprises: pulling structured data out of invoices and receipts, triaging and drafting email replies, and summarizing documents — all running on local, offline AI rather than a cloud API. That is still exactly the goal. What has changed since the proposal is mostly **how much of the groundwork is now actually running**, and, just as importantly, **what I have learned by trying to make it work on a real, unglamorous laptop instead of a clean cloud environment**.

Rather than describe the plan again, I want to use this report to walk through what has actually been built, the problems that came up along the way, and where I think the honest gaps are before this becomes something a real SME could rely on.

2 System Architecture

The application follows a fairly standard three-layer shape: a React front end, a FastAPI backend, and an AI processing pipeline sitting behind it that chains together YOLO (for finding **where** the relevant information is on a page), PaddleOCR (for reading **what** it says), and — eventually — a locally-hosted Ollama model (for turning raw text into clean, structured fields). A SQLite database persists everything: documents, invoices, emails, and settings. Nothing calls out to an external API at any point; this was a deliberate constraint from the start, both to keep the system usable for an SME without ongoing cloud costs, and because a good deal of the documents involved — invoices, exams, internal paperwork — are exactly the kind of thing that shouldn't leave the machine they were uploaded on.

3 Getting the Foundation Running

The front end — the dashboard, the documents view, invoices, the email manager, a voice page, and settings, along with a working dark theme — was already scaffolded going into this phase, so the first real piece of work was building the backend it had nothing to talk to yet. I set up a FastAPI application with the SQLite schema laid out in the original plan (documents, invoices, emails,

summaries, voice commands, and settings), and a `/api/system/status` endpoint that genuinely checks — not just assumes — whether YOLO, PaddleOCR, Ollama, and the voice component are actually available on the machine at that moment. That status check turned out to be more useful than I expected, because almost everything that follows in this report started with that endpoint quietly telling me something was broken.

4 Building the Document Intelligence Pipeline

This is the part of the project that took the most unglamorous, trial-and-error effort, and I think it is worth documenting honestly rather than glossing over.

4.1 Environment problems before a single document was processed

The very first obstacle had nothing to do with machine learning at all: the system's Python version (3.14) turned out to be too new for PaddlePaddle, the library PaddleOCR is built on, which currently only ships wheels up to Python 3.13. I resolved this by using uv to install an isolated Python 3.12 just for the backend, without touching the system Python used everywhere else. A related, easy-to-miss trap was that installing YOLO's dependency, PyTorch, would by default pull in roughly two gigabytes of NVIDIA CUDA libraries — completely wasted on this machine, which only has Intel integrated graphics. Installing the CPU-only build instead avoided that entirely.

Even after the environment was sorted out, PaddleOCR itself did not run cleanly out of the box: its current CPU inference path has a real bug in how it uses the oneDNN acceleration backend, which crashed on the very first detection call. The fix was a single environment flag to disable that acceleration path, but finding it meant reading through PaddleOCR's own source rather than relying on outdated examples online — the library's API had also changed substantially between major versions, so the OCR integration had to be written against what the installed version actually does, not what older tutorials describe.

None of these were interesting machine learning problems. They were exactly the kind of "why won't this even install" problems that do not show up in a project plan, but ate a genuinely significant share of the time in this phase.

4.2 The pipeline itself

With the environment stable, the actual processing pipeline came together as: an uploaded PDF or image is rendered to a raster image if needed, YOLO looks at that image and proposes bounding boxes for the regions it recognizes, PaddleOCR reads text out of the **whole** page for a general transcript, and — this is the part I am most pleased with — PaddleOCR is run a second time on each individual region YOLO detected, so that the extracted text comes back already tied to its semantic label (for example, the text inside a detected "subject" box arrives as `{"subject": "Calculus II"}` not just as an anonymous line somewhere in a wall of text). Structured field extraction for invoices specifically (invoice number, vendor, date, total) is currently done with a regex-based heuristic, clearly marked in the code as a placeholder for the Ollama-based structuring that Phase 3 of the plan calls for and has not been started yet.

5 A Practical Detour: Teaching YOLO to Read a Real Document

The stock YOLO model only knows the 80 everyday object categories it was trained on — cars, people, chairs — and has no idea what an invoice number field looks like. Making it useful here means fine-tuning it on examples of the actual document type, with hand-drawn boxes around the

fields that matter. Real invoices, however, are surprisingly hard to collect in bulk as a student, both because of confidentiality and because there simply aren't many lying around. Lebanese University exam papers turned out to be a genuinely useful stand-in: they are structurally similar documents — a header with a logo, a subject field, a date, boxed question regions — that I actually had access to in quantity, which let me validate the entire detect-then-read technique end to end before ever worrying about whether it generalizes to invoices specifically.

I defined eleven region classes for an exam page (logo, question, subject, date, faculty, duration, semester, department, university name, and two smaller administrative fields), and hand-labeled twenty-three exam images using **make-sense.ai** — deliberately self-hosted from its source rather than used through the public site, since these images do carry students' names and IDs and I did not want that leaving my machine.

Fine-tuning a small YOLOv8 model on that first batch of twenty-three images took about twelve minutes on this laptop's CPU and stopped early after seventy-three epochs once it stopped improving. The headline number — an overall mAP50 of roughly 0.68 — is honestly not something I would read too much into on its own, since the validation set was only three images. What convinced me the approach was actually working was testing it on a real exam page the model had never seen during training: it correctly located the question region and, through the crop-then-OCR pipeline described above, accurately read out an entire multi-part question about Dijkstra's algorithm and graph centrality — genuinely tied to the right label, not just floating text somewhere in a transcript. That was the first moment this felt like a working system rather than a collection of parts that each worked in isolation.

Two of the eleven classes — the smallest, thinnest label boxes — did not detect reliably at all in this first run, which is exactly what I would expect from having only about twenty training examples to learn from; I expect that to improve directly with more labeled data. To make sure "more data" is actually possible, I went and pulled four full departments' worth of past exams — Business Computer Science, Communications Engineering, Industrial Engineering, and Civil Engineering — which came out to 755 additional exam PDFs. I rendered the first page of each into an image, bringing the total labeling pool to 784 images. I have deliberately not tried to label all of that at once; a curated, department-balanced subset of a few hundred is a far more realistic next labeling session, and better for teaching the model a variety of layouts than exhausting one department first.

6 Bringing It Into the Actual App

Once the pipeline and the trained model were working from the command line, I wired all of it into the real application rather than leaving it as a standalone script. Document upload now genuinely runs through the full detect-and-read pipeline in the background rather than showing placeholder data, and the interface shows live status as a document moves from pending to processing to processed. I added an in-browser camera capture flow — including a "live recognition" mode that runs the fast, OCR-free half of the pipeline repeatedly against the live camera feed, drawing bounding boxes over the video in close to real time as you move the camera over a document, similar to a barcode scanner locking onto a target. After a document is processed, its detail view now shows the actual analyzed image with the detected regions drawn on it, in the same style as the labeling tool, alongside the OCR'd text grouped by which region it came from.

7 Honest Assessment

I want to be direct about where this stands rather than overstate it. The plumbing — upload, detect, read, structure, store, display — genuinely works end to end, on real documents, including live camera input. That is a meaningful milestone. But the trained model behind it right now is a proof of concept, trained on twenty-three images, not a production-ready detector; two of its eleven classes are weak, and its accuracy needs to be re-established once trained on a larger, more diverse batch. Field structuring is currently a regex heuristic standing in for the LLM-based extraction the plan calls for in the next phase, and email handling and voice input have not been started at all yet. CPU-only training was tolerable at this small scale (minutes, not hours) but will not stay that way as the dataset grows into the hundreds of images; I have access to a laptop with a dedicated GPU and intend to move training there once the next labeling batch is ready.

8 What's Next

The immediate next steps, in the order I plan to tackle them, are: labeling a larger, department-balanced batch of the newly collected exam images; retraining on that larger set, ideally on GPU hardware; and then starting Phase 3 of the original plan — connecting Ollama so that field extraction and email drafting are handled by an actual language model instead of the regex placeholder currently in place. Voice input remains the last major piece and is intentionally scheduled last, as originally planned, since it depends the least on the document pipeline being solid first.

9 Closing Note

Most of what I would call "progress" this cycle was not glamorous — it was getting a machine learning stack to actually install and run correctly on ordinary hardware, working around real bugs in third-party libraries, and being careful about where student and business data is allowed to go. I think that work was necessary and worth documenting, even though none of it shows up as a feature on a roadmap. The system now does something real, end to end, on real documents; the next cycle is about making it good rather than making it exist.